EXTSM

In the low-level standard of COBOL compilers (such as Micro Focus, COBOL-IT, or isCOBOL), the interface with the sort module is formally established using the following statement:
CALL “EXTSM” USING OP-CODE, SORT-FCD.

The **FCD (File Control Description)** not only defines the structure of the record or keys (Key Definition Block), but also contains internal pointers, status flags, and I/O stream descriptors (the physical files involved).
The various operational scenarios and the specific combination of **opcodes** and **FCD** configurations vary depending on the case:

Scenario 1: “File-to-File” Direct Sort (USING / GIVING)
This is the scenario in which **EXTSM autonomously manages the opening, reading, and writing** of physical files, bypassing the processing of individual records.

• **Opcodes and data stream sent by COBOL:**
1. **Opcode 01 (INIT)**: COBOL compiles the FCD by inserting pointers to the physical names (or descriptors) of the input and output files, as well as the lengths and positions of the keys. It sends opcode 01.
2. **Bypassing intermediate opcodes**: EXTSM sees in the FCD that the input/output streams are populated internally. The external engine independently opens and reads the input file, performs the sort, and writes directly to the output file.
3. **Opcode 05 (TERM):** COBOL sends the termination code to free up sort memory and delete the working files.

• **FCD Status**: The FCD contains valid references to both files (Input and Output).

Scenario 2: Manual Loading and Unloading (INPUT PROCEDURE / OUTPUT PROCEDURE)

The COBOL program wants to manipulate the data. Consequently, the physical files are not passed to EXTSM; instead, the runtime system reads and writes one record at a time.

• **Codes and Flow Sent by COBOL**:

1. **Opcode 01 (INIT)**: The FCD is initialized only with the logical record specifications of the sort file (SD) and the key definitions. The pointers to I/O files in the FCD are set to NULL or zero.

2. **Opcode 02 (PUT)**: Sent cyclically by COBOL (in conjunction with the RELEASE statement). With each call, COBOL passes Opcode 02, the FCD (which maintains the session state), and the buffer of the single record retrieved or created.

3. **Opcode 03 (SORT)**: When the INPUT PROCEDURE ends, COBOL sends Opcode 03 to sort the records accumulated by EXTSM.

4. **Opcode 04 (GET)**: Sent cyclically by COBOL (at the RETURN statement). EXTSM populates the record buffer and updates the end-of-file flags in the FCD when no more records are available.

5. **Opcode 05 (TERM)**: Sent at the end of the COBOL statement to deallocate resources.

Scenario 3: Mixed Scenario (e.g., INPUT PROCEDURE / GIVING)

This is used when the input data needs to be processed on a record-by-record basis (e.g., filtered), but the output must be written directly to a file at high speed without going through COBOL.

- **Codes and Flow Sent from COBOL:**

1. **Opcode 01 (INIT)**: The FCD is populated to specify that input will arrive in records (Input File = Null), but a valid descriptor/path for the output file (GIVING) is provided.

2. **Opcode 02 (PUT)**: COBOL sends the records one by one (RELEASE).

3. **Opcode 03 (SORT)**: COBOL declares the end of input. At this point, EXTSM sorts the data and, upon detecting the presence of the output file in the FCD, writes the final file directly to disk.

4. **Opcode 05 (TERM)**: Standard session termination.

- (Note: If the scenario were reversed (USING / OUTPUT PROCEDURE), Opcode 01 would contain the input file in the FCD; EXTSM would read the file on its own, and COBOL would use Opcodes 03, 04 (GET), and 05 to manually extract the data.)

Scenario 4: Merge Operation (MERGE)

The MERGE statement combines already sorted files into a single stream.

- **Codes and Flow Sent by COBOL**:

1. **Opcode 06 (MERGE-INIT)** or an initialization variant: COBOL configures the FCD by defining the keys and specifying the list of multiple input files to be merged.

2. **I/O Flow**: When paired with GIVING, EXTSM reads the files and writes the output in bulk. When paired with OUTPUT PROCEDURE, COBOL will use Opcode 04 (GET) to retrieve the merged records one at a time.

3. **Opcode 05 (TERM)**: Closes the merge operation.

Summary of the COBOL Communication ➡ EXTSM

Scenario	Initial Opcode	Input Handling (FCD)	Output Handling (FCD)	Flow Opcode
File-to-File	01 (INIT)	Managed by EXTSM (File located in FCD)	Managed by EXTSM (File located in FCD)	None (PUT/GET Bypass)
Total Procedure	01 (INIT)	Managed by COBOL (File missing in FCD)	Managed by COBOL (File missing in FCD)	Loop 02 (PUT) ➡ 03 (SORT) ➡ Loop 04 (GET)
Mixed (Input Proc + Giving)	01 (INIT)	Managed by COBOL (File missing in FCD)	Managed by EXTSM (File located in FCD)	Loop 02 (PUT) ➡ 03 (SORT)

extsmtst5.cbl

```
*> ------------------------------------------------------------
*> Set environment Variable
*> -- export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH
*> ------------------------------------------------------------
*> EXTSMTST5 - prepares the FCD3s for 3 input files, 1 output file, and
*> the FCD for SORT, then calls EXTSM.  No subprograms:
*> FCDSET is a SECTION.
*>
*> Usage:  extsmtst           -> SORT   (function code X"FA01")
*>       extsmtst MERGE     -> MERGE  (function code X"FA40")
*>
*> The file xfhfcd3.cpy (with constants 78) is included ONCE,
*> in LINKAGE as CUR-FCD: it is a "view" that is moved with
*> SET ADDRESS OF CUR-FCD TO ... to the FCD to be processed.
*>
*> Pointer chain:
*>   SORT-FCD-AREA --FCD-FILDEF-ADDRESS--> FDB (pointer table)
*>                                          |-> FILE-FCD(1) in 1
*>                                          |-> FILE-FCD(2) in 2
*>                                          |-> FILE-FCD(3) in 3
*>                                          '-> FILE-FCD(4) out
*>   SORT-FCD-AREA --FCD-KEY-DEF-ADDRESS--> KDB-AREA (one key)
*> -------------------- ---------------------------------------
*> ------------------------------------------------------------ ----
*> Positional arguments, separated by spaces:
*>   extsmtst                    -> using=3 give=1 (default)
*>   extsmtst 0 0                -> RELEASE / RETURN
*>   extsmtst 3 0                -> input file, RETURN
*>   extsmtst 0 1                -> RELEASE, output file
*>   extsmtst 3 1                -> everything to file (default)
*>   extsmtst 3 1 MERGE          -> Same as above, but MERGE (FA40)
*> ------------------------------------------------------------
 IDENTIFICATION DIVISION.
 PROGRAM-ID. EXTSMTST5.
 DATA DIVISION.
 WORKING-STORAGE SECTION.

*> function code documentati (byte alto sempre X"FA")
 78  FN-SORT-INIT        VALUE 1.
 78  FN-MERGE-INIT       VALUE 64.
*> FAF3
 78  FN-RELEASE          VALUE 243.
*> FA10
 78  FN-END-INPUT        VALUE 16.
*> FAF5
 78  FN-RETURN           VALUE 245.
*> FA88
 78  FN-TERMINATE        VALUE 136.

*> N-USING/N-GIVING erano costanti 78 (fisse): ora sono variabili,
*> lette da riga di comando, cosi' si possono provare tutte le
*> combinazioni della tabella senza ricompilare.
 78  N-FILES-MAX         VALUE 4.
 78  REC-LEN             VALUE 80.
 01  N-USING             PIC 9 COMP-5 VALUE 3.
 01  N-GIVING            PIC 9 COMP-5 VALUE 1.
```

```
 01  FUNC-CODE.
     05 FN-HI            PIC X COMP-X.
     05 FN-LO            PIC X COMP-X.

*> aree di memoria delle FCD (216 byte = LENGTH OF FCD3)
 01  SORT-FCD-AREA       PIC X(216).
 01  FILE-FCD-TAB.
     05 FILE-FCD         PIC X(216) OCCURS N-FILES-MAX.

*> File Definition Block: tabella di puntatori a 64 bit
 01  FDB.
     05 FDB-PTR          USAGE POINTER OCCURS N-FILES-MAX.

 01  FILE-NAMES.
     05 FILLER           PIC X(64)
              VALUE
     "../files/extsm_input1.dat".
     05 FILLER           PIC X(64)
              VALUE
     "../files/extsm_input2.dat".
     05 FILLER           PIC X(64)
              VALUE
     "../files/extsm_input3.dat".
     05 FILLER           PIC X(64)
              VALUE
     "../files/extsm_output.dat".
 01  FILE-NAMES-T REDEFINES FILE-NAMES.
     05 FILE-NAME        PIC X(64) OCCURS N-FILES-MAX.

*> un buffer record per file (layout: CODICE 10 + IMPORTO 6 + ..)
 01  REC-BUFS.
     05 REC-BUF          PIC X(REC-LEN) OCCURS N-FILES-MAX.

*> ------------------------------------------------------------
*> KEY DEFINITION BLOCK (layout da struct C KDB / EXTKEY):
*>   intestazione 14 byte: kdbLen[2] filler[4] nkeys[2] filler[6]
*>   KDB_KEY 16 byte: count[2] offset[2] keyFlags compFlags
*>                    sparse reserved[9]
*>   componenti EXTKEY da 10 byte: desc, type, pos[4], len[4]
*> Da verificare: origine di KS-COMP-OFFSET (assunta: dall'inizio
*> del KDB) e pos 0-based.
*> ------------------------------------------------------------
 01  KDB-AREA.
     05 KDB-HDR.
        10 KDB-LENGTH        PIC 9(4) COMP-X.
        10 FILLER            PIC X(4).
        10 KDB-KEY-COUNT     PIC 9(4) COMP-X.
        10 FILLER            PIC X(6).
     05 KDB-KEYSPEC.
        10 KS-COMP-COUNT     PIC 9(4) COMP-X.
        10 KS-COMP-OFFSET    PIC 9(4) COMP-X.
        10 KS-KEYFLAGS       PIC X COMP-X.
        10 KS-COMPFLAGS      PIC X COMP-X.
        10 KS-SPARSE         PIC X.
        10 FILLER            PIC X(9).
     05 KDB-COMP OCCURS 2.
        10 CS-FLAG1          PIC X COMP-X.
        10 CS-FLAG2          PIC X COMP-X.
```

```
          10 CS-OFFSET         PIC X(4) COMP-X.
          10 CS-LENGTH         PIC X(4) COMP-X.

 01  WS-ACTION           PIC X.
 01  WS-MODE             PIC X(16).
 01  WS-I                PIC 9(4) COMP-5.
 01  WS-NX          PIC 9(4) COMP-5.
 01  WS-NAME-LEN         PIC 9(4) COMP-5.
 01  WS-OPEN             PIC 9(4) COMP-5.
 01  WS-ARG-1            PIC X(16).
 01  WS-ARG-2            PIC X(16).
 01  WS-ARG-3             PIC X(16).
 01  WS-MERGE            PIC 9 COMP-5.
 01  WS-NFILES           PIC 9(4) COMP-5.

*> --- record area e dati per RELEASE/RETURN (USING=0/GIVING=0) --
 78  N-SRC-REC           VALUE 8.
 01  REC-AREA            PIC X(REC-LEN).
 01  SRC-DATA.
     05 FILLER.
        10 FILLER PIC X(10) VALUE "ROSSI".
        10 FILLER PIC 9(6) VALUE 100.
        10 FILLER PIC X(64) VALUE "primo".
     05 FILLER.
        10 FILLER PIC X(10) VALUE "BIANCHI".
        10 FILLER PIC 9(6) VALUE 50.
        10 FILLER PIC X(64) VALUE "secondo".
     05 FILLER.
        10 FILLER PIC X(10) VALUE "ROSSI".
        10 FILLER PIC 9(6) VALUE 300.
        10 FILLER PIC X(64) VALUE "terzo".
     05 FILLER.
        10 FILLER PIC X(10) VALUE "VERDI".
        10 FILLER PIC 9(6) VALUE 10.
        10 FILLER PIC X(64) VALUE "quarto".
     05 FILLER.
        10 FILLER PIC X(10) VALUE "BIANCHI".
        10 FILLER PIC 9(6) VALUE 50.
        10 FILLER PIC X(64) VALUE "quinto-dup".
     05 FILLER.
        10 FILLER PIC X(10) VALUE "ROSSI".
        10 FILLER PIC 9(6) VALUE 100.
        10 FILLER PIC X(64) VALUE "sesto-dup".
     05 FILLER.
        10 FILLER PIC X(10) VALUE "VERDI".
        10 FILLER PIC 9(6) VALUE 99.
        10 FILLER PIC X(64) VALUE "settimo".
     05 FILLER.
        10 FILLER PIC X(10) VALUE "BIANCHI".
        10 FILLER PIC 9(6) VALUE 75.
        10 FILLER PIC X(64) VALUE "ottavo".
 01  SRC-DATA-T REDEFINES SRC-DATA.
     05 SRC-REC           PIC X(REC-LEN) OCCURS N-SRC-REC.
 01  WS-RETCOUNT         PIC 9(4) COMP-5.

 LINKAGE SECTION.
*> unica inclusione della copy: vista mobile sulla FCD corrente
 01  CUR-FCD.
     COPY "xfhfcd3.cpy".
```

```
 01  CUR-NAMEV           PIC X(64).

 PROCEDURE DIVISION.

 MAIN-SEC SECTION.
 MAIN-P.

     MOVE 250 TO FN-HI                   *> X"FA"
     MOVE 0 TO WS-MERGE
     ACCEPT WS-MODE FROM COMMAND-LINE
     PERFORM PARSE-ARGS

     DISPLAY "Input: USING=" N-USING " GIVING=" N-GIVING
             " merge=" WITH NO ADVANCING
     IF WS-MERGE = 1
          DISPLAY "yes"
     ELSE
          DISPLAY "no"
     END-IF

     PERFORM CHECK-SIZES
     PERFORM BUILD-FILE-FCDS
     PERFORM BUILD-KDB
     PERFORM BUILD-SORT-FCD
     PERFORM CALL-EXTSM
     PERFORM SHOW-RESULTS
     goback.
     STOP RUN.

*> ------------------------------------------------------------
*> Argomenti posizionali, separati da spazio:
*>   extsmtst                   -> using=3 give=1 (default)
*>   extsmtst 0 0               -> RELEASE / RETURN
*>   extsmtst 3 0               -> file input, RETURN
*>   extsmtst 0 1               -> RELEASE, file output
*>   extsmtst 3 1               -> tutto su file (default)
*>   extsmtst 3 1 MERGE         -> come sopra, ma MERGE (FA40)
*> ------------------------------------------------------------
 PARSE-ARGS SECTION.
 PARSE-ARGS-P.
     MOVE FUNCTION UPPER-CASE(WS-MODE) TO WS-MODE
     IF WS-MODE = SPACES
        EXIT SECTION
     END-IF
     UNSTRING WS-MODE DELIMITED BY ALL SPACES
          INTO WS-ARG-1 WS-ARG-2 WS-ARG-3
     END-UNSTRING
     IF WS-ARG-1 NOT = SPACES
        AND FUNCTION TRIM(WS-ARG-1) IS NUMERIC
        MOVE FUNCTION NUMVAL(FUNCTION TRIM(WS-ARG-1)) TO N-USING
     END-IF
     IF WS-ARG-2 NOT = SPACES
        AND FUNCTION TRIM(WS-ARG-2) IS NUMERIC
        MOVE FUNCTION NUMVAL(FUNCTION TRIM(WS-ARG-2)) TO N-GIVING
     END-IF
     IF WS-ARG-1(1:5) = "MERGE" OR WS-ARG-2(1:5) = "MERGE"
        OR WS-ARG-3(1:5) = "MERGE"
        MOVE 1 TO WS-MERGE
     END-IF
```

```
     .

 CHECK-SIZES SECTION.
 CHECK-SIZES-P.
*>   le aree X(216) devono coincidere con la FCD3
     IF LENGTH OF CUR-FCD NOT = LENGTH OF FILE-FCD(1)
        OR LENGTH OF CUR-FCD NOT = LENGTH OF SORT-FCD-AREA
        DISPLAY "FCD3 = " LENGTH OF CUR-FCD
                " byte (" LENGTH OF FILE-FCD(1)
                ")"
        MOVE 1 TO RETURN-CODE
        STOP RUN
     END-IF.

*> ------------------------------------------------------------
*> FCDSET - prepara ("I") o mostra ("S") la FCD del file WS-I.
*> Parametri: WS-ACTION, WS-I.  Si richiama con PERFORM FCDSET.
*> I rami sono inline: una SECTION viene eseguita tutta di
*> seguito, senza salti tra paragrafi.
*> ------------------------------------------------------------
 FCDSET SECTION.
 FCDSET-P.
     SET ADDRESS OF CUR-FCD TO ADDRESS OF FILE-FCD(WS-I)

     EVALUATE WS-ACTION
        WHEN "I"
*>         LOW-VALUES e non INITIALIZE: INITIALIZE salta i
*>         FILLER della copy (aree riservate)
           MOVE LOW-VALUES TO CUR-FCD
           MOVE LENGTH OF CUR-FCD      TO FCD-LENGTH
           MOVE fcd--version-number    TO FCD-VERSION
           MOVE fcd--sequential-org    TO FCD-ORGANIZATION
           MOVE fcd--sequential-access TO FCD-ACCESS-MODE
           MOVE fcd--open-closed       TO FCD-OPEN-MODE
           MOVE fcd--recmode-fixed     TO FCD-RECORDING-MODE
           MOVE fcd--dev-normal        TO FCD-DEVICE-FLAG
           MOVE fcd--ans85-status      TO FCD-STATUS-TYPE
*>         nome file: puntatore + lunghezza, NON terminato da NUL
           MOVE FUNCTION LENGTH(FUNCTION TRIM(FILE-NAME(WS-NX)))
                                       TO WS-NAME-LEN
           MOVE WS-NAME-LEN            TO FCD-NAME-LENGTH
           SET FCD-FILENAME-ADDRESS
                              TO ADDRESS OF FILE-NAME(WS-NX)
*>         area record
           SET FCD-RECORD-ADDRESS
                              TO ADDRESS OF REC-BUF(WS-NX)
           MOVE REC-LEN TO FCD-MIN-REC-LENGTH
                           FCD-MAX-REC-LENGTH
                           FCD-CURRENT-REC-LEN
        WHEN "S"
           IF FCD-NAME-LENGTH > 0
              AND FCD-FILENAME-ADDRESS NOT = NULL
              SET ADDRESS OF CUR-NAMEV TO FCD-FILENAME-ADDRESS
              DISPLAY CUR-NAMEV(1:FCD-NAME-LENGTH)
                  WITH NO ADVANCING
           END-IF
           MOVE FCD-OPEN-MODE TO WS-OPEN
           DISPLAY " org=" FCD-ORGANIZATION
                   " open-mode=" WS-OPEN
```

```
                  " reclen=" FCD-CURRENT-REC-LEN
                  " status=" WITH NO ADVANCING
           EVALUATE TRUE
              WHEN FCD-STATUS-KEY-1 = LOW-VALUE
                 DISPLAY "(non valorizzato)"
              WHEN FCD-STATUS-KEY-1 = "9"
                 DISPLAY "9/" FCD-BINARY
              WHEN OTHER
                 DISPLAY FCD-STATUS-KEY-1 FCD-STATUS-KEY-2
           END-EVALUATE
     END-EVALUATE.

 BUILD-FILE-FCDS SECTION.
 BUILD-FILE-FCDS-P.
*>   prima gli USING, poi i GIVING; se USING+GIVING=0 non c'e'
*>   nessuna FCD di file da preparare (solo RELEASE/RETURN)
     COMPUTE WS-NFILES = N-USING + N-GIVING
     IF WS-NFILES = 0
        EXIT SECTION
     END-IF
     MOVE "I" TO WS-ACTION
     PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-NFILES
        IF WS-I <= N-USING
           MOVE WS-I TO WS-NX
        ELSE
           COMPUTE WS-NX = 3 + (WS-I - N-USING)
        END-IF
        PERFORM FCDSET
        SET FDB-PTR(WS-I) TO ADDRESS OF FILE-FCD(WS-I)
     END-PERFORM.

 BUILD-KDB SECTION.
 BUILD-KDB-P.
*>   Una chiave, due componenti:
*>     1) bytes 0-9   alfanumerico, ascendente
*>     2) bytes 10-15 numerico DISPLAY senza segno, DESCENDENTE
     MOVE LOW-VALUES TO KDB-AREA
     MOVE LENGTH OF KDB-AREA TO KDB-LENGTH
     MOVE 1                  TO KDB-KEY-COUNT
     MOVE 2                  TO KS-COMP-COUNT
     COMPUTE KS-COMP-OFFSET = LENGTH OF KDB-HDR
                            + LENGTH OF KDB-KEYSPEC
*> KEY_DUPS x"40"
     MOVE 64  TO KS-KEYFLAGS
     MOVE 0   TO KS-COMPFLAGS
*> ascendente
     MOVE 0   TO CS-FLAG1(1)
*> non numerico
     MOVE 0   TO CS-FLAG2(1)
     MOVE 1   TO CS-OFFSET(1)
     MOVE 10  TO CS-LENGTH(1)
*> x"40" = DESCENDING
     MOVE 64  TO CS-FLAG1(2)
*> x"80" = NUMERIC DISPLAY
     MOVE 128 TO CS-FLAG2(2)
     MOVE 11  TO CS-OFFSET(2)
     MOVE 6   TO CS-LENGTH(2).
     display "---------------------------------"
     display "KS-KEYFLAGS = " KS-KEYFLAGS
```

```
     display "KS-COMPFLAGS= " KS-COMPFLAGS
     display "Key 1: "
     display " CS-FLAG1(1)= "  CS-FLAG1(1)
     display " CS-FLAG2(1)= "  CS-FLAG2(1)
     display "CS-OFFSET(1)= " CS-OFFSET(1)
     display "CS-LENGTH(1)= " CS-LENGTH(1)
     display "Key 2: "
     display " CS-FLAG1(2)= "  CS-FLAG1(2)
     display " CS-FLAG2(2)= "  CS-FLAG2(2)
     display "CS-OFFSET(2)= " CS-OFFSET(2)
     display "CS-LENGTH(2)= " CS-LENGTH(2)
     display "--------------------------------"
     .


 BUILD-SORT-FCD SECTION.
 BUILD-SORT-FCD-P.
     SET ADDRESS OF CUR-FCD TO ADDRESS OF SORT-FCD-AREA
     MOVE LOW-VALUES TO CUR-FCD
     MOVE LENGTH OF CUR-FCD      TO FCD-LENGTH
     MOVE fcd--version-number    TO FCD-VERSION
     MOVE fcd--sequential-org    TO FCD-ORGANIZATION
     MOVE fcd--sequential-access TO FCD-ACCESS-MODE
     MOVE fcd--open-closed       TO FCD-OPEN-MODE
     MOVE fcd--recmode-fixed     TO FCD-RECORDING-MODE

*>   quanti file: gli N-USING sono i primi della tabella,
*>   poi seguono gli N-GIVING
     MOVE N-USING  TO FCD-USE-FILES
     MOVE N-GIVING TO FCD-GIVE-FILES

     MOVE REC-LEN TO FCD-MIN-REC-LENGTH
                     FCD-MAX-REC-LENGTH
                     FCD-CURRENT-REC-LEN

     SET FCD-KEY-DEF-ADDRESS TO ADDRESS OF KDB-AREA
     SET FCD-COL-SEQ-ADDRESS TO NULL       *> nessuna collating
     SET FCD-FILDEF-ADDRESS  TO ADDRESS OF FDB.

**
 CALL-EXTSM SECTION.
 CALL-EXTSM-P.
     IF WS-MERGE = 1
        MOVE FN-MERGE-INIT TO FN-LO
        DISPLAY "-> FA40 (MERGE)"
     ELSE
        MOVE FN-SORT-INIT TO FN-LO
        DISPLAY "-> FA01 (SORT)"
     END-IF
     CALL "EXTSM" USING FUNC-CODE SORT-FCD-AREA
     PERFORM SHOW-SORT-STATUS

     IF N-USING = 0
*>      RELEASE di ogni record della tabella dati di prova,
*>      poi FA10: fine input, EXTSM ordina
        SET ADDRESS OF CUR-FCD TO ADDRESS OF SORT-FCD-AREA
        SET FCD-RECORD-ADDRESS TO ADDRESS OF REC-AREA
        PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > N-SRC-REC
           MOVE SRC-REC(WS-I) TO REC-AREA
```

```
            MOVE REC-LEN TO FCD-CURRENT-REC-LEN
            MOVE FN-RELEASE TO FN-LO
            DISPLAY "-> FAF3 (RELEASE) rec " WS-I WITH NO ADVANCING
            CALL "EXTSM" USING FUNC-CODE SORT-FCD-AREA
            DISPLAY " status " FCD-STATUS-KEY-1 FCD-STATUS-KEY-2
         END-PERFORM
         MOVE FN-END-INPUT TO FN-LO
         DISPLAY "-> FA10 (fine input)"
         CALL "EXTSM" USING FUNC-CODE SORT-FCD-AREA
         PERFORM SHOW-SORT-STATUS
      END-IF

      IF N-GIVING = 0
*>       RETURN in ciclo fino a status 10 (fine dati)
         SET ADDRESS OF CUR-FCD TO ADDRESS OF SORT-FCD-AREA
         SET FCD-RECORD-ADDRESS TO ADDRESS OF REC-AREA
         MOVE 0 TO WS-RETCOUNT
         PERFORM UNTIL 1 = 0
            MOVE FN-RETURN TO FN-LO
            CALL "EXTSM" USING FUNC-CODE SORT-FCD-AREA
            IF FCD-STATUS-KEY-1 = "1"
               EXIT PERFORM
            END-IF
            IF FCD-STATUS-KEY-1 NOT = "0"
               DISPLAY "-> FAF5 (RETURN) status " FCD-STATUS-KEY-1
                       FCD-STATUS-KEY-2
               EXIT PERFORM
            END-IF
            ADD 1 TO WS-RETCOUNT
            DISPLAY "-> FAF5 (RETURN) rec " WS-RETCOUNT ": "
                    REC-AREA(1:REC-LEN)
         END-PERFORM
         DISPLAY "total record RETURN: " WS-RETCOUNT
      END-IF

*>    CORREZIONE: Questa sezione ora e' FUORI dall'IF.
*>    Viene eseguita SEMPRE alla fine di ogni tipologia di ordinamento.
      MOVE FN-TERMINATE TO FN-LO
      DISPLAY "-> FA88 (termina)"
      CALL "EXTSM" USING FUNC-CODE SORT-FCD-AREA
      PERFORM SHOW-SORT-STATUS.

 SHOW-SORT-STATUS.
      SET ADDRESS OF CUR-FCD TO ADDRESS OF SORT-FCD-AREA
      DISPLAY "   status FCD SORT = " WITH NO ADVANCING
      EVALUATE TRUE
         WHEN FCD-STATUS-KEY-1 = LOW-VALUE
            DISPLAY "(NULL)"
         WHEN FCD-STATUS-KEY-1 = "9"
            DISPLAY "9/" FCD-BINARY
         WHEN OTHER
            DISPLAY FCD-STATUS-KEY-1 FCD-STATUS-KEY-2
      END-EVALUATE.

 SHOW-RESULTS SECTION.
 SHOW-RESULTS-P.
*>    lo status di ogni file (se presenti) va letto nella sua FCD

      IF WS-NFILES = 0
```

```
           EXIT SECTION
        END-IF
        DISPLAY "Stato delle FCD dei file dopo la chiamata:"
        MOVE "S" TO WS-ACTION
        PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-NFILES
           IF WS-I <= N-USING
              MOVE WS-I TO WS-NX
              DISPLAY "  [" WS-I "] USING"
           ELSE
              COMPUTE WS-NX = 3 + (WS-I - N-USING)
              DISPLAY "  [" WS-I "] GIVING"
           END-IF
           PERFORM FCDSET
        END-PERFORM.
    END PROGRAM EXTSMTST5.
```